Appendix G. Modern Observability: Statistics and ML

This appendix provides a deeper look into the statistical and machine learning models we use for the intelligent alerting systems described in Chapter 9. While the chapter focuses on the strategic shift away from static thresholds, here we explore the technical underpinnings of how we create alerts that are both precise and adaptive.

A Statistical Approach for Error Rates

To move away from static alerting thresholds, we see an opportunity to apply a more rigorous statistical approach. Consider one of the most common signals: error rates. For any given request, the outcome is binary—it either succeeds or fails. This scenario isn't just a random stream of events; it is a classic statistical pattern that we can model with a *binomial distribution*.

The first step is to teach our system what "normal" looks like for each individual service. Instead of a one-size-fits-all threshold, we establish a dynamic baseline error probability by analyzing the last two weeks of traffic. Without diving too much into mathematical formalisms, we use Wilson's confidence interval to describe the expected distribution of errors, and if we go above the maximum expected error range, we alert (many statistical methods could be used here with similar results, but we have found this one to be robust and easy to implement).[1]

This approach is naturally volume-aware. For a low-traffic service, the acceptable range of errors is wider, preventing alerts from a few random failures. For a high-volume service, the range becomes proportionally much tighter, making it sensitive to even minor upticks in the error rate that could impact thousands of users. This automatically solves the problem our old static thresholds never could, freeing SREs from tuning alerts and letting them trust that when an alarm does sound, it signifies a real deviation from the norm.

It is relatively easy to implement and can be done in multiple ways, from expressing the binomial test directly in a monitoring system query language to building a periodic job that computes the upper bounds of the confidence interval and publishes them as a baseline metric for comparison.

Handling Cyclical Metrics with Machine Learning

In many services, error rates are low and typically stable over the lifetime of the service. But other metrics, like resource usage (e.g., RAM, CPU), may not be. There may be daily or weekly patterns of peaks and troughs that correspond to how the service is used. How can we alert on unusual behavior for such services? Using simple statistical methods like the one described in the previous section won't work.

Instead, we can train a machine learning model that predicts what we should expect now based on what was observed in the past. If what we see does not match what the model expects, we alert. Models can learn periodic behavior and provide confidence bounds. They can be trained for specific types of underlying data or be general time-series models, such as TimesFM.

ML is incredibly powerful but comes with two important considerations: explainability and cost. Not every anomaly is problematic (for example, a one-off event can cause a sudden traffic spike even when the system as a whole is operating as intended), but when it is, it can be difficult to explain why. In addition, ML models are significantly more expensive than statistical models. Therefore, we can't simply deploy them everywhere and forget about other monitoring approaches.

A Cost-Effective Application: Post-Alert Analysis

One area where we have found ML to be especially effective is post-alert analysis of anomalies. In Chapter 10, we talked about investigation dashboards that are constructed dynamically in response to an alert. At this point, we have a strong prior signal (the alert fired), so now every anomaly is a potential clue. The cost of using an ML model is justified, and also much smaller, as we evaluate only when things are broken instead of at regular intervals.

[1] The Wilson interval score is discussed in the sidebar "Using Binomial Testing".

Previous chapter
< F. The Mathematics of Latency Alerting

Next chapter
Index >